sltoo – Integrating Requirements into CI/CD

Kristoffer Nordström

September 1, 2021

info@sltoo.dev

sltoo – Integrating Requirements into CI/CD

Kristoffer Nordström

September 1, 2021

info@sltoo.dev

- Snapshot: wie sieht es aus
  - More pictures
- Contribution of mine hervorstreichen
- Mehr Beispiele
  - Personify Eugen from Marketing (?)

- Snapshot: wie sieht es aus
  - More pictures
- Contribution of mine hervorstreichen
- Mehr Beispiele
  - Personify Eugen from Marketing (?)

Motivation

Motivation

It's an honour to open this topic's session.

When talking about requirements, we should also define the requirements of the problem we're trying to solve. For all requirements there's a anecdote.

Storytime

- Fully automated traceability matrix
- Consistent and up-to-date documents from the source
- Store (requirements') meta-information with code

Storytime
- Fully automated traceability matrix
- Consistent and up-to-date documents from the source
- Store (requirements') meta-information with code

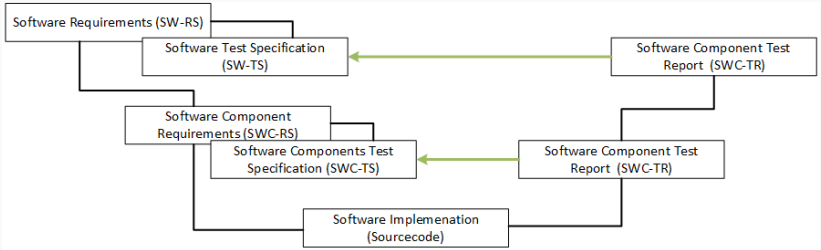
- It's fine manually. Requirements change find minimal delta-test? Change more than once. Hence we want this fully automated.
- Review multiple documents for a different contractor. No consistent set of documents were provided
- Jon Holt's term in introduction to MBSE: Documents are a live *view* of the system (not pretty pictures). CI/CD for software, why not for documents?
- It came with one document that contained all requirements and use-cases. It appeared to have been exported from some web-based tool. All the information therein is lost
- Also: Too technical, interface with management/business side
- 4'

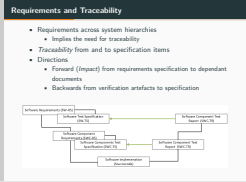
Theory

Theory

Requirements and Traceability

- Requirements across system hierarchies
  - Implies the need for traceability
- *Traceability* from and to specification items
- Directions
  - Forward (*Impact*) from requirements specification to dependant documents
  - Backwards from verification artefacts to specification





Here Traceability only from items/issues

Traceability

- *Requirement A*
  - Red button to shut down system
- *Implementation a* says implemented A
  - Traceability can be automated
  - Machine-readable
- What if *A* changes?
  - *A* knows nothing of *a*

Traceability
Requirement A
Red button to shut down system
Implementation a says implemented A
Traceability can be automated
Machine-readable
What if A changes?
A knows nothing of a
PANIC

Traceability

- *Requirement A*
  - NGreen button with large friendly letters: don't panic
- *Implementation a* says implemented `A`
  - Traceability can be automated
  - Machine-readable
- What if *A* changes?
  - *A* knows nothing of *a*

Traceability
Requirement A
NGreen button with large friendly letters: don't panic
Implementation a says implemented A
Traceability can be automated
Machine-readable
What if A changes?
A knows nothing of a
DON'T PANIC

Proposed Solution

- *Requirement A-1.0* says do `A`
- *Implementation a* says implemented `A-1.0`
  - *A-1.0* changes to *A-2.0* do `Ã`
  - Use hashes instead of semantic versioning
  - Calculated automatically

DON'T PANIC

Proposed Solution
Requirement A-1.0 says do A
Implementation a says implemented A-1.0
A-1.0 changes to A-2.0 do Ã
Use hashes instead of semantic versioning
Calculated automatically
DON'T PANIC

Example Requirement

VCD Writer Inputs SW-AS-501
The output from *AggregatorInterface* can be fed into the *VCD Writer* to create an identical output file.
Rationale: Make the process as transparent as possible.
Depends on: SW-AS-500

Id:	SW-AS-501	Priority:	0.0	Owner:	development
Invented on:	2020-05-30	Invented by:	default	Status:	passed
Class:	detailable	Hash:	f8d68d11		

Hash is calculated over *Name*, *Description* and *Verification Method*.
Rationale is only informative

```
$ sha256sum "${Name}${Description}${VerifMethod}"
```


Example Requirement
VCD Writer Inputs SW-AS-501
The output from AggregatorInterface can be fed into the VCD Writer to create an identical output file.
Rationale: Make the process as transparent as possible.
Depends on: SW-AS-500
Id: SW-AS-501 Priority: 0.0 Owner: development
Invented on: 2020-05-30 Invented by: default Status: passed
Class: detailable Hash: f8d68d11
Hash is calculated over Name, Description and Verification Method.
Rationale is only informative
$ sha256sum "${Name}${Description}${VerifMethod}"

Testing the Example Requirement

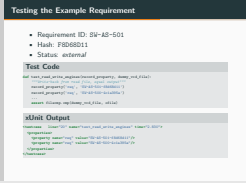
- Requirement ID: `SW-AS-501`
- Hash: `F8D68D11`
- Status: *external*

Test Code

```python
def test_read_write_engines(record_property, dummy_vcd_file):
    """Write-back from read file, equal output"""
    record_property('req', 'SW-AS-501-f8d68d11')
    record_property('req', 'SW-AS-500-4c1a395a')
    ...
    assert filecmp.cmp(dummy_vcd_file, ofile)
```

xUnit Output

```xml
<testcase   line="20" name="test_read_write_engines" time="2.830">
  <properties>
    <property name="req" value="SW-AS-501-f8d68d11"/>
    <property name="req" value="SW-AS-500-4c1a395a"/>
  </properties>
</testcase>
```

Traceability Matrix

Traceability Matrix

Table A.1: Traceability Matrix Table

Req. ID	Status	UT
SW-AS-1	passed	passed
SW-AS-100	open	open
SW-AS-101	open	open
SW-AS-102	open	open
SW-AS-200	passed	passed
SW-AS-201	passed	passed
SW-AS-202	passed	passed
SW-AS-300	passed	passed
SW-AS-301	open	open
SW-AS-302	open	open
SW-AS-500	passed	passed
SW-AS-501	passed	passed

Traceability Matrix
Traceability Matrix
Table A.1: Traceability Matrix Table
Req. ID Status UT
SW-AS-1 passed passed
SW-AS-100 open open
SW-AS-101 open open
SW-AS-102 open open
SW-AS-200 passed passed
SW-AS-201 passed passed
SW-AS-202 passed passed
SW-AS-300 passed passed
SW-AS-301 open open
SW-AS-302 open open
SW-AS-500 passed passed
SW-AS-501 passed passed

Integrating Requirements into CI/CD

- Integration for every output document
- Match *open* and/or *failed* issues
- Example for *failed* issues

```
$ bash -ec 'test "$(grep -c failed \
    arch/artifacts/tracematrix.tex)" -eq "0"'
```


Integrating Requirements into CI/CD
- Integration for every output document
- Match *open* and/or *failed* issues
- Example for *failed* issues

$ bash -ec 'test "$(grep -c failed \
arch/artifacts/tracematrix.tex)" -eq "0"'

Document Baseline

Every document has a its own version tag

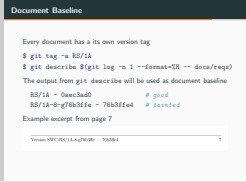
```
$ git tag -a RS/1A
$ git describe $(git log -n 1 --format=%H -- docs/reqs)
```

The output from `git describe` will be used as document baseline

```
RS/1A - 0aec3ad0              # good
RS/1A-8-g76b3ffe - 76b3ffe4   # tainted
```

Example excerpt from page 7

Version SWC-RS/1A-8-g76b3ffe — 76b3ffe4 7



This is all nice and dandy; but how do we communicate with people for whom git clone is too much to ask?

Business Interface

Business Interface

Excel Workflow

- Non-tech people define the requirements
- Familiarity / Ease-of-use
- Consistency of Documents
  - Works if all you've got is Office and E-Mail
- INSERT IMAGE HERE

Excel Workflow
- Non-tech people define the requirements
- Familiarity / Ease-of-use
- Consistency of Documents
  - Works if all you've got is Office and E-Mail
- INSERT IMAGE HERE

Conclusion

Conclusion

Storytime Revisited

- Requirements shipped with code ✓
  - Including relational meta-information
- Traceability matrix automated ✓
- Continuously updated documentation ✓
  - Document Versioning (baselining) ✓

Storytime Revisited
- Requirements shipped with code ✓
  - Including relational meta-information
- Traceability matrix automated ✓
- Continuously updated documentation ✓
  - Document Versioning (baselining) ✓

Your Jira workspace will be gone

Questions

Questions

Appendix

Excel Support Rationale

- Good-enough GUI
- Understood by everyone
- The *Truth* is still in your repository
  - Import from Excel to repository
  - Export every build to a new Excel file
  - Templating for branding

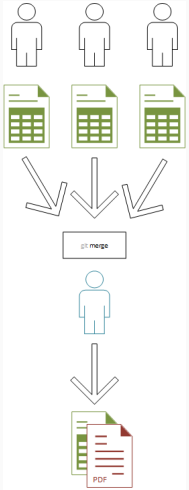


Figure 1: Workflow

Excel Support Rationale
Good-enough GUI
Understood by everyone
The *Truth* is still in your repository
Import from Excel to repository
Export every build to a new Excel file
Templating for branding
Figure 1: Workflow

rmtoo – Introductions

An introduction presentation into *rmtoo* and with more details.

rmtoo – Introductions
An introduction presentation into *rmtoo* and with more details.

Traceability Rationale

- Traceability for the given requirements
- Bring code and documentation into same repository
- Integrate into build-system
  - Detect upstream changes to requirements
  - Quickly identify affected code-regions
- No silver bullet for verification

Traceability Rationale
- Traceability for the given requirements
- Bring code and documentation into same repository
- Integrate into build-system
  - Detect upstream changes to requirements
  - Quickly identify affected code-regions
- No silver bullet for verification

Results

The status *external* will yield the following results:

- *open*
  - No matching requirement ID
- *passed*
  - Matching requirement ID
  - All hashes match
  - Unit-tests passed
- *failed*
  - Matching requirement ID
  - Some/all hashes didn't match, or
  - Unit-tests haven't passed

Results

The status *external* will yield the following results:

- *open*
  - No matching requirement ID
- *passed*
  - Matching requirement ID
  - All hashes match
  - Unit-tests passed
- *failed*
  - Matching requirement ID
  - Some/all hashes didn't match, or
  - Unit-tests haven't passed

Installation

Traceability features are in the beta releases.

```
$ pip3 install sltoo>=25.1.0b3
$ wget https://kown7.github.io/pymergevcd/assets/template_p
```


Installation
Traceability features are in the beta releases.
$ pip3 install sltoo>=25.1.0b3
$ wget https://kown7.github.io/pymergevcd/assets/template_p

Future Developements

- Write Parser for *Test Reports* ✓
- Documents with the correct identifier automatically solve the specification
  - Document Formats:
    - docx (maybe with pandoc)
    - LaTeX✓
    - Text

Future Developements
Write Parser for Test Reports ✓
Documents with the correct identifier automatically solve the specification
Document Formats:
docx (maybe with pandoc)
LaTeX✓
Text

Final Thoughts

- Never test against your requirements
- Always write some form of test specification

Final Thoughts
- Never test against your requirements
- Always write some form of test specification

Licensing

This work is licensed under a Creative Commons "Attribution-NonCommercial-ShareAlike 3.0 Unported" license.

Licensing

This work is licensed under a Creative Commons "Attribution-NonCommercial-ShareAlike 3.0 Unported" license.

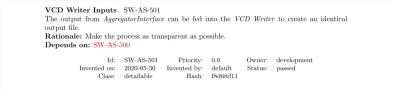
Baseline

- Based on rmtoo
  - File-based requirements tracking tool
  - Requirements stored in text-files
  - group requirements by topics
  - document and requirements LaTeXbased
    - Customizable with templates
- Manage with your favourite VCS
  - Merge changes from various sources
- Excel import/export
- Traceability

Baseline
- Based on rmtoo
  - File-based requirements tracking tool
  - Requirements stored in text-files
  - group requirements by topics
  - document and requirements LaTeXbased
    - Customizable with templates
- Manage with your favourite VCS
  - Merge changes from various sources
- Excel import/export
- Traceability

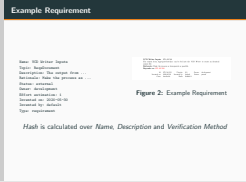
Example Requirement

```
Name: VCD Writer Inputs
Topic: ReqsDocument
Description: The output from ...
Rationale: Make the process as ...
Status: external
Owner: development
Effort estimation: 1
Invented on: 2020-05-30
Invented by: default
Type: requirement
```



VCD Writer Inputs SW-AS-501
The output from *AggregatorInterface* can be fed into the *VCD Writer* to create an identical output file.
Rationale: Make the process as transparent as possible.
Depends on: SW-AS-500

Id:	SW-AS-501	Priority:	0.0	Owner:	development
Invented on:	2020-05-30	Invented by:	default	Status:	passed
Class:	detailable	Hash:	f8d0bd11		

Figure 2: Example Requirement

Hash is calculated over *Name*, *Description* and *Verification Method*